

Agora-Style RTC Service Platform

A self-hosted WebRTC SaaS architecture for tenant apps, external users, RTC tokens, rooms, signaling, chat, usage, billing, and future SFU-based media scaling.

Multi-company
SaaS

Client API keys

External user sync

RTC token API

SFU-ready

Platform Owner

Runs the RTC SaaS. Creates companies, packages, apps, usage, billing, and incidents.



Client Company

Owns the app and users. Integrates with your REST APIs and SDK. Requests tokens.



End User

Never signs up on your site. Joins RTC rooms inside the client app using a short-lived token.

Prepared as a build roadmap and technical product brief

Version 1.0 - May 31, 2026

00 Document Map

A quick way to read this PDF if you only need the product direction first.

This PDF converts the new direction into a practical SaaS product plan. The core idea is to build an Agora-like RTC service, but with your own self-hosted WebRTC infrastructure instead of using Agora SDKs or Agora APIs.

Section	What it explains
1. Product Model	Who uses the platform: platform owner, client company, and end user.
2. Architecture	How the control plane, APIs, token service, signaling, media, and usage systems connect.
3. Admin Dashboard	How the dashboard should become company-first, not demo-first.
4. Client Setup	What information is needed to onboard a client company manually.
5. External User API	How client companies sync their own users as shadow users.
6. RTC Token API	How the Agora-style token flow should work.
7. Room API	How client companies create, list, update, disable, and delete rooms.
8. Developer Docs	What the Developer section should contain for integration.
9. RTC Engine Upgrade	How to start with native WebRTC and later add SFU scaling.
10. Usage and Billing	What to track for manual billing and package limits.
11. Build Order	The recommended first implementation milestones.

Best first build target

Start with the company-first admin dashboard, client API key authentication, external user sync API, RTC token API, Developer docs page, and room management per company. Add SFU/media scaling after the SaaS control plane is clean.

01 Core Product Model

The platform should behave like an RTC infrastructure provider, not like a public chat app.

The platform owner sells RTC capability to client companies. The client companies already have their own users, apps, websites, and business logic. Your system should provide the communication layer: rooms, tokens, signaling, media connectivity, chat, usage tracking, billing visibility, and admin controls.

Actor	Owns	What they do in the platform
Platform owner	RTC SaaS platform, companies, packages, billing, incidents	Creates companies, manages apps/API keys, monitors usage, suspends tenants, edits packages, and controls system-level operations.
Client company	Their own app, website, users, payments, business logic	Integrates your RTC service by API/SDK. Their backend requests RTC tokens. Their frontend joins rooms.
End user	Only their account inside the client app	Does not register on your site. They join RTC rooms through a short-lived token generated for their external user ID.

Simple mental model

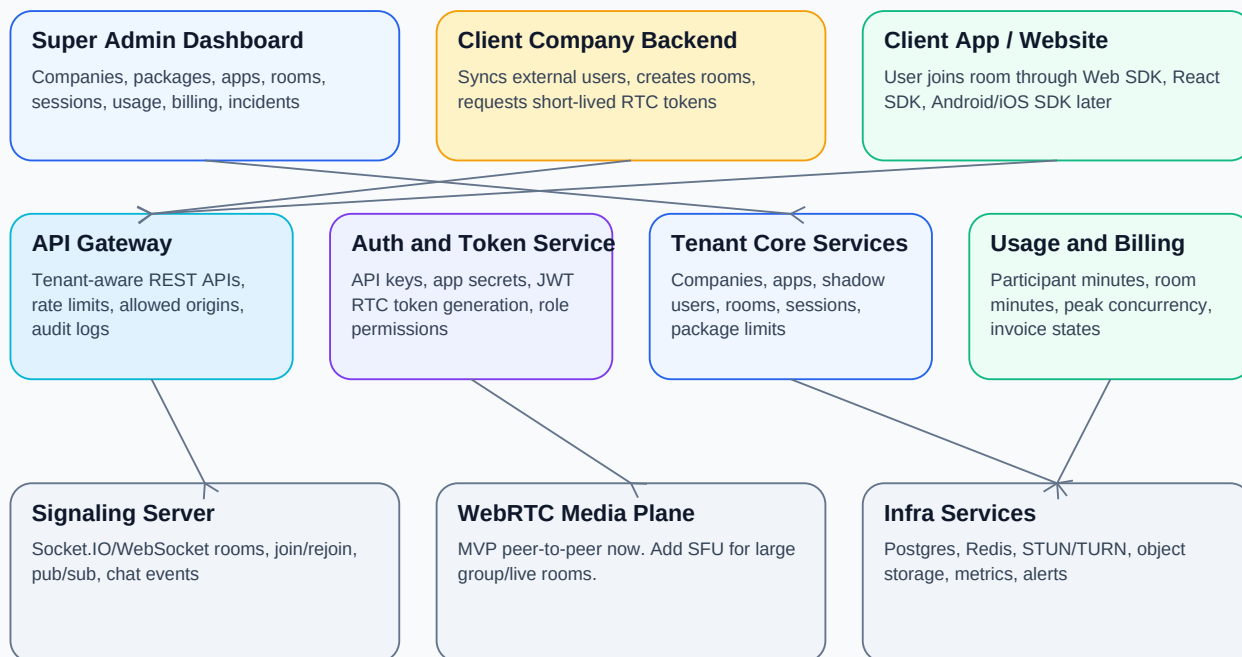
Your platform is the RTC infrastructure layer. Client companies bring users and products. Your APIs give them secure room access and your dashboard gives you tenant, usage, and billing control.

02 Agora-Like Architecture

Use the Agora pattern, but with your own token, signaling, and media infrastructure.

Target Architecture: Agora-like control plane with self-hosted media plane

The control plane handles tenants, auth, rooms, tokens, usage, billing, and admin. The media plane handles RTC delivery.



Capability	MVP implementation	Future scale implementation
Tenant apps	Company apps with app key, app secret, allowed origins, and package limit	Multiple apps per company with app-level quotas, environment separation, and key rotation.
Auth and tokens	Client API key authenticates backend calls. RTC JWT authorizes room join/publish/chat.	Scoped app secrets, tenant-level rate limits, webhook signing, token revocation, audit logs.
Signaling	WebSocket or Socket.IO room signaling for join, offer/answer, ICE, chat, presence.	Redis pub/sub or NATS for multi-node signaling and cross-region routing.
Media	Native peer-to-peer WebRTC for small rooms and MVP testing.	SFU layer using mediasoup, Janus, or a self-hosted LiveKit-style architecture for larger group rooms/live rooms.
Usage	Track session start/end and participant minutes by tenant/app/room.	Real-time metering, overage estimates, package enforcement, alerting, invoice export.

Important technical decision

Peer-to-peer WebRTC is acceptable for MVP and small rooms, but it is not enough for Agora-like multi-user scaling. The architecture should be SFU-ready from day one, even if the SFU is added in a later phase.

03 Phase 1: Multi-Company Admin Dashboard

The dashboard should start with companies, because this is a SaaS control panel.

The current direction should move away from a demo RTC website and become a platform owner console. The first screen should show all client companies, their status, package, usage, active rooms, and operational health.

Page	Purpose	Main actions
Company list	Show all tenants at a glance.	Create company, search/filter, view status, package, active rooms, usage, payment state.
Company detail	Central dashboard for one tenant.	Open tabs for Overview, Apps, Rooms, Users, Usage, Billing, Settings.
Company users	Show shadow users synced from client systems.	Search by external_user_id, view metadata, active sessions, disable user access.
Company rooms	Show all rooms for that tenant.	Create/open/delete/disable room, view participants, room type, features, status.
Active sessions	Operational live view.	See active users, room duration, device/client info, reconnect status, network stats later.
Apps/API keys	Manage app identities.	Generate API key, rotate secret, set allowed origins, suspend app.
Package/pricing	Manual commercial control.	Assign package, edit limits, overage rate, manual payment status.
Usage/billing	Billing visibility.	Participant minutes, room minutes, peak concurrency, billing estimate, invoice status.

Recommended dashboard layout: left navigation focuses on Companies. The center panel shows the selected company dashboard. The top action bar includes Create company, Generate API key, Suspend, and Edit package.

Tab	What admin sees
Overview	Company status, package, payment state, active rooms, active users, usage this month, incidents.
Apps	App keys, app type, allowed domains/origins, app status, key rotation date.
Rooms	All rooms, room types, active/disabled state, max seats, feature flags, participants.
Users	Shadow users synced from the client, metadata, last seen, session history.
Usage	Participant minutes, room minutes, peak concurrent users, active rooms, overage estimate.
Billing	Package, manual payment status, unpaid/paid/overdue/manual confirmed, invoice notes.
Settings	Company profile, contact details, security, webhook URLs, suspension controls.

04 Phase 2: Client Company Registration and Setup

Payment stays manual, but package and pricing data must be editable.

Client company setup should collect enough commercial, technical, and support information to operate the tenant safely. This does not mean end users register on your platform. It means the company itself is registered as a tenant.

Setup field group	Fields to store
Company identity	Company name, legal name, country, website/app URL, app type.
Business contact	Business email, phone, primary contact name, primary contact email.
Messaging contact	Telegram, WhatsApp, Discord.
Commercial package	Package, package limits, manual payment status, invoice status, custom pricing notes.
Security settings	Allowed domains/origins, app type: web, Android, iOS, server, API key status.
Operational status	Active, suspended company, suspended app, disabled rooms, incident notes.

Manual payment does not mean no billing system

Even if payments are collected manually, the system should still calculate usage and show invoice status. Manual confirmed, unpaid, paid, and overdue are enough for the first version.

05 Phase 3: External User API

Client users should be shadow users, not users forced into your signup system.

This is the most important technical change. Client companies should sync their existing users into your platform through an API. Your database stores a tenant-scoped shadow user linked to the company app and external user ID. This gives you usage, session, moderation, and permission control without owning the client company login system.

```
POST /api/client/users/sync
Authorization: Bearer CLIENT_API_KEY
Content-Type: application/json

{
  "external_user_id": "imo_user_123",
  "name": "Julian",
  "avatar_url": "https://client-app.com/avatar.jpg",
  "gender": "male",
  "metadata": {
    "vip_level": 3
  }
}
```

System behavior	Details
Create or update	If tenant_id + app_id + external_user_id does not exist, create it. Otherwise update name, avatar, gender, and metadata.
Tenant isolation	The same external_user_id can exist in multiple companies because it is unique only inside tenant/app scope.
No password	Shadow users do not need passwords because authentication is controlled by the client company backend.
Useful for reporting	Sessions, room joins, chat moderation, usage, and bans can be linked to shadow user records.

Database uniqueness rule

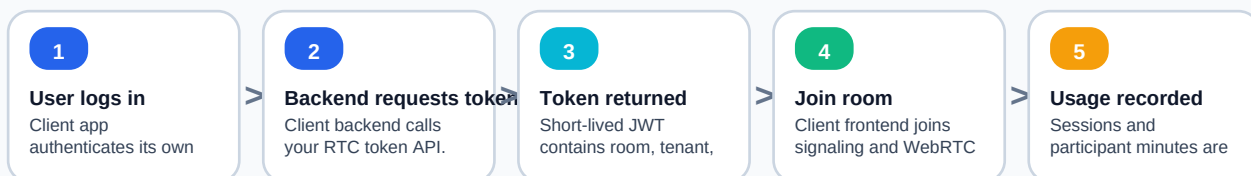
Use a unique constraint like tenant_id + app_id + external_user_id. Never make external_user_id globally unique across all tenants.

06 Phase 4: RTC Token API

This is the real Agora-style flow: app backend requests a short-lived token.

The client company backend calls your token endpoint after verifying its own user. Your platform checks the company, app, package, room, user, role, permissions, and status. Then it returns a short-lived RTC token for the client frontend to join the room.

Runtime Flow: user stays inside the client company app



```

POST /api/client/rtc/token
Authorization: Bearer CLIENT_API_KEY
Content-Type: application/json

{
  "external_user_id": "imo_user_123",
  "room_id": "room_456",
  "role": "publisher",
  "permissions": ["join", "publish_audio", "publish_video", "chat"]
}

{
  "rtc_token": "short_lived_jwt",
  "expires_in": 900,
  "room": {
    "id": 456,
    "signaling_room": "tenant_12_room_456"
  }
}
  
```

Token claim	Why it matters
tenant_id	Prevents cross-company access.
app_id	Connects usage and room access to the correct client app.
shadow_user_id / external_user_id	Maps the RTC session to the client company user.
room_id and signaling_room	Limits token to one room/channel.
role	Controls publisher, subscriber, host, moderator, or audience behavior.
permissions	Controls join, publish_audio, publish_video, screen_share, chat, kick, mute.
exp / iat	Keeps token short-lived and safe. 15 minutes is a good default.

07 Phase 6: Room Service API

Client companies need to create and manage rooms from their own backend.

Room APIs make the platform useful for real products like IMO-style messaging, dating apps, education rooms, live rooms, and PK rooms. The room object should be tenant-scoped and app-scoped.

```
POST /api/client/rooms
GET /api/client/rooms
PATCH /api/client/rooms/:id
POST /api/client/rooms/:id/disable
DELETE /api/client/rooms/:id
```

Room field	Recommended meaning
name	Human-friendly room name used by admin and client app.
type	audio, video, live, or pk.
privacy	public, private, invite_only, or tenant-defined.
max_seats	Maximum participants or seats allowed for this room.
chat_enabled	Whether in-room chat is enabled.
gifts_enabled	Whether virtual gifts are enabled.
screen_share_enabled	Whether participants can share screens.
beauty_filter_enabled	Whether frontend SDK should expose beauty filter feature.
ai_guard_enabled	Whether future moderation/AI safety checks should run.
status	active, disabled, deleted, or archived.

Admin override

Platform owner should always be able to disable a room from the admin console, even if the room was created by a client company API.

08 Phase 5: SDK and Developer Docs

The product needs an integration layer that feels easy for client companies.

A Developer section is essential because client companies will not use the dashboard every day. Their engineers need clear API docs, token examples, SDK samples, allowed-origin configuration, webhooks, and a test console.

Developer section page	Content
Quick start	Create app, copy API key, sync user, request token, join room.
REST API docs	External user sync, RTC token, rooms, sessions, webhooks.
Web SDK sample	Plain JavaScript example for joining audio/video room.
React sample	Reusable hook/component: useRtcRoom(token), publish camera/mic, leave room.
Token flow	Backend-only token request, short-lived token, permissions, expiration.
Room flow	Create room, request token, join signaling room, publish/subscription flow.
Webhook docs	room.started, room.ended, participant.joined, participant.left, usage.updated.

Developer section page	Content
Error codes	Invalid API key, app suspended, room disabled, origin blocked, package limit reached.
Allowed origins	How to restrict web apps by domain and environment.
Test console	Try sync user, create room, request token, and copy example code.

```
// Example client flow
1. User logs into the client company app.
2. Client backend calls POST /api/client/rtc/token.
3. Client frontend receives rtc_token.
4. Client frontend joins the signaling room.
5. WebRTC connection starts.
6. Your platform records usage and active session data.
```

09 Phase 7: RTC Engine Upgrade Path

Keep native WebRTC for MVP, but design the platform so SFU can be added cleanly.

The control plane should not care whether media delivery is peer-to-peer or SFU. This separation is what makes the platform future-proof. The APIs, rooms, users, tokens, sessions, and billing can be built now. The media plane can evolve later.

Feature	MVP	SFU-ready future
Signaling server	WebSocket or Socket.IO for room join, offers, answers, ICE candidates, leave events.	Clustered signaling with Redis/NATS, shard rooms, resilient reconnect logic.
STUN/TURN	Use self-hosted TURN for NAT traversal reliability.	Monitor TURN usage and cost, add region selection.
Small rooms	Peer-to-peer WebRTC can work for 1:1 and small rooms.	Move group rooms to SFU when participant count or quality requires it.
Publish/subscribe	Role controls decide who can publish audio/video/chat.	Server-side SFU permissions and subscription quality layers.
Reconnect/rejoin	Basic reconnect and token refresh.	Session resume, media re-subscribe, connection quality tracking.
Network quality	Basic browser WebRTC stats.	Admin charts, packet loss, bitrate, RTT, jitter, publisher quality.
Moderation	Mute/kick/ban in signaling and admin.	SFU-enforced mute, server-side recording/moderation later.
Recording	Not needed first unless required.	Add room recording after SFU is stable.

Recommended engineering rule

Build a media adapter interface now. Example: `MediaProvider = p2p | mediasoup | janus | livekit_style`. The dashboard and APIs should call your own service layer, not direct media-server code.

10 Phase 8: Admin Billing and Usage

Manual payment is fine, but usage must be measured like a real SaaS platform.

The platform owner needs visibility into usage even if payments are collected outside the product. Usage data also enables package limits, overage estimates, abuse detection, and renewal conversations.

Metric	Definition	Used for
Participant minutes	Sum of minutes each participant spends in a room.	Primary usage billing and package limits.
Room minutes	Total active room duration.	Operational analytics and room-level cost visibility.
Peak concurrent users	Highest simultaneous active users in a period.	Capacity planning and package tier decisions.
Active rooms	Number of rooms currently active.	Admin live monitoring and incident response.
Recording minutes later	Minutes of server-side recording if enabled later.	Optional add-on billing.
Package limit	Included participant minutes, rooms, users, apps, or concurrency.	Overage and suspension logic.

Metric	Definition	Used for
Invoice status	Unpaid, paid, overdue, manual confirmed.	Manual commercial workflow.

Package concept	Example editable fields
Starter	1 app, 10 active rooms, 5,000 participant minutes, basic support.
Growth	3 apps, 100 active rooms, 50,000 participant minutes, higher concurrency.
Enterprise	Custom apps, custom participant minutes, custom SLA, manual contract.
Overage	Per 1,000 participant minutes, per recording minute later, custom notes.

11 Core Data Model

These entities make the SaaS control plane work cleanly.

Entity	Key fields	Notes
companies	id, name, legal_name, country, status, package_id, payment_status	Main tenant record. Everything should be tenant-scoped.
company_contacts	company_id, name, email, phone, telegram, whatsapp, discord	Keeps business/support contact data flexible.
client_apps	company_id, app_key, app_secret_hash, app_type, status, allowed_origins	One company can have multiple apps/environments.
api_keys	app_id, key_hash, scopes, status, last_used_at, rotated_at	Use hashed keys. Support rotation and revocation.
shadow_users	company_id, app_id, external_user_id, name, avatar_url, metadata	Unique by company_id + app_id + external_user_id.
rooms	company_id, app_id, external_room_id, type, privacy, max_seats, feature_flags, status	Can be created by API or admin.
rtc_tokens	company_id, app_id, shadow_user_id, room_id, role, permissions, expires_at	Optional audit table. Do not store raw token if not needed.
rtc_sessions	company_id, app_id, room_id, shadow_user_id, started_at, ended_at, duration_seconds	Basis for participant minutes and active sessions.
usage_daily	company_id, app_id, date, participant_minutes, room_minutes, peak_concurrent	Pre-aggregated billing and dashboard data.
billing_invoices	company_id, period, amount_estimate, status, manual_confirmed_by	Manual billing workflow.
webhooks	company_id, app_id, url, secret_hash, events, status	Send events to client company backend.
audit_logs	actor_type, actor_id, company_id, action, target_type, target_id, metadata	Needed for admin actions and tenant security.

Tenant isolation rule

Every runtime object should include company_id and usually app_id. Query filters must always apply tenant scope before returning rooms, users, sessions, or usage.

12 Security, Reliability, and Controls

This platform sells infrastructure, so tenant isolation and operational controls matter from the first version.

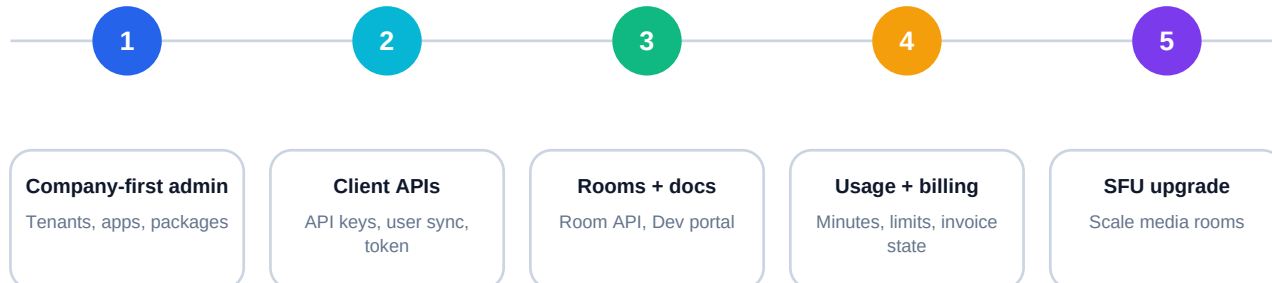
Area	Requirement
API key safety	Store only hashed API keys. Show the raw key only once when generated. Support rotation and suspension.
Allowed origins	For web apps, validate origin against allowed domains before issuing or accepting RTC access.
Short-lived RTC token	Default expiration around 15 minutes. Refresh token flow can be added later.
Role permissions	Do not rely only on frontend checks. Server must encode role and permissions in the token.
Package limits	Optionally block token creation when company is suspended, app is suspended, room is disabled, or hard limit is reached.

Area	Requirement
Audit logs	Record create company, generate key, suspend, disable room, edit package, manual payment confirmation.
Rate limits	Rate-limit API key endpoints to prevent abuse and accidental loops.
Incident controls	Admin should be able to suspend a company, app, room, or user quickly.
Metrics	Track API latency, token failures, active sessions, signaling errors, TURN errors, and media reconnects.

13 What to Build First

The highest-value path is to build the SaaS control plane before deep media scaling.

Recommended Build Sequence



Priority	Build item	Why it comes first
1	Company-first admin dashboard	It changes the product from demo RTC site to SaaS control panel.
2	Client API key authentication	Required before any client company can integrate safely.
3	External user sync API	Allows client users to remain inside client company apps.
4	RTC token API	Creates the Agora-style join model and unlocks secure room access.
5	Room management per company	Lets client companies create/manage rooms through API and lets admin control them.
6	Developer docs page	Makes the product integratable and reduces support confusion.
7	Usage and billing visibility	Enables manual selling, package limits, and platform owner monitoring.
8	SFU/media scaling	Add when rooms need scale beyond peer-to-peer small rooms.

Most practical MVP definition

A working MVP is not a huge live-streaming media system. A working MVP is a tenant dashboard plus secure APIs where a client backend can sync a user, request a token, create a room, and the client frontend can join a small WebRTC room while your platform tracks usage.

14 Suggested API Route Map

A clean route structure keeps platform-owner APIs separate from client-company APIs.

API area	Routes
Admin companies	GET /api/admin/companies, POST /api/admin/companies, GET /api/admin/companies/:id, PATCH /api/admin/companies/:id
Admin company control	POST /api/admin/companies/:id/suspend, POST /api/admin/companies/:id/activate
Admin apps/API keys	POST /api/admin/companies/:id/apps, POST /api/admin/apps/:id/api-keys, POST /api/admin/apps/:id/suspend
Admin rooms	GET /api/admin/companies/:id/rooms, POST /api/admin/rooms/:id/disable, DELETE /api/admin/rooms/:id

API area	Routes
Admin usage/billing	GET /api/admin/companies/:id/usage, GET /api/admin/companies/:id/billing, PATCH /api/admin/invoices/:id
Client users	POST /api/client/users/sync, GET /api/client/users/:external_user_id
Client rooms	POST /api/client/rooms, GET /api/client/rooms, PATCH /api/client/rooms/:id, POST /api/client/rooms/:id/disable, DELETE /api/client/rooms/:id
Client RTC	POST /api/client/rtc/token, POST /api/client/rtc/session/start, POST /api/client/rtc/session/end
Developer test console	POST /api/dev/test/sync-user, POST /api/dev/test/create-room, POST /api/dev/test/token

Naming recommendation

Use /api/admin/* for platform owner operations and /api/client/* for client-company backend integration. Never mix admin privileges into client API keys.

15 Implementation Checklist

A practical task list for the first development sprint sequence.

Workstream	Tasks
Tenant foundation	Create companies, packages, apps, API keys, allowed origins, status fields.
Admin UI shell	Left nav: Companies. Detail tabs: Overview, Apps, Rooms, Users, Usage, Billing, Settings.
API key middleware	Authenticate CLIENT_API_KEY, resolve company/app, reject suspended company/app.
Shadow user sync	Create/update shadow user by tenant/app/external_user_id.
Room service	Create/list/update/disable/delete tenant rooms with feature flags.
Token service	Generate short-lived JWT with tenant, app, user, room, role, permissions, expiration.
Signaling validation	Require token on WebSocket join. Validate room, tenant, app, role, and permissions.
Session tracking	Start/end sessions and calculate duration. Handle disconnect/reconnect safely.
Usage aggregation	Daily participant minutes, room minutes, peak concurrent users.
Developer docs	Quick start, API docs, sample token flow, React/Web SDK examples, error codes.
Admin controls	Suspend company/app, disable room, disable user access, rotate API key, edit package.
Observability	Basic logs, API errors, signaling errors, active sessions, usage counters.

Done condition for MVP: A client company can be created, receive an API key, sync a user, create a room, request an RTC token, join a WebRTC room from a sample frontend, and see usage appear in the admin dashboard.

16 Appendix: Example Error Codes and Webhooks

These make the integration feel mature from the beginning.

Error code	Meaning
invalid_api_key	The client API key is missing, invalid, revoked, or malformed.
company_suspended	The tenant company is suspended. Token and room APIs should fail.
app_suspended	The specific client app is suspended.
origin_not_allowed	The web origin is not in the app allowed origins list.
room_disabled	The requested room exists but is disabled by admin or client.
room_not_found	The requested room does not exist in this tenant/app scope.
user_not_synced	The external user must be synced before token generation, unless auto-create is enabled.
permission_denied	Requested role or permissions are not allowed for this app, room, or package.
package_limit_reached	The company has reached a hard package limit.

```
Webhook events to support later:
- room.started
- room.ended
- participant.joined
- participant.left
- participant.reconnected
- usage.updated
```

```
- room.disabled  
- package.limit_warning  
- package.limit_reached
```

Final direction

Build the control plane first: company dashboard, client auth, user sync, RTC token, rooms, docs, and usage. Then upgrade the media plane with SFU when customers need larger rooms.